

System Design and Edge Deployment of a Cognitive Strategy and Diagnostics Assistant for WEC and IMSA Hypercar/GTP Campaigns

The execution of elite sports car racing campaigns in the FIA World Endurance Championship (WEC) and the IMSA WeatherTech SportsCar Championship demands rapid, high-precision decision-making under severe intellectual property and non-disclosure constraints. Factory and customer racing programs utilizing Le Mans Hypercar (LMH) or Le Mans Daytona hybrid (LMDh) platforms operate in an environment where proprietary engineering configurations cannot be exposed to cloud-based processing layers. To bridge this gap, this report presents a local, edge-deployed cognitive chatbot assistant and mathematical modeling architecture designed for real-time trackside strategic optimization, diagnostic troubleshooting, and predictive analysis.

System Architecture for Local Edge Deployment

Operating a strategy and diagnostics assistant from the pit wall of circuits like Spa-Francorchamps or Daytona requires absolute data confidentiality, low latency, and independence from external network connections. Direct cloud connections represent a severe vulnerability, risking the exposure of confidential vehicle configurations, such as internal combustion engine thermal maps, hybrid state-of-charge tables, and proprietary software logs. Furthermore, standard cloud-based APIs can fail entirely in remote circuit environments with poor or saturated network infrastructure.

To eliminate these risks, this architecture is designed to execute locally within the engineering team's trackside hardware environment. The setup leverages Anaconda Navigator or a virtualized Conda environment in VS Code, running a fully containerized local machine learning stack.

The user interface is built with Streamlit, providing a lightweight, low-latency front-end that runs locally. The back-end is driven by FastAPI, which routes queries to a stateful LangGraph orchestrator. The model execution layer is managed via Ollama, utilizing quantized local Small Language Models (SLMs) such as Qwen 2.5 Coder or Mistral 7B to perform code parsing, database querying, and natural language generation.

[User Natural Language Query]

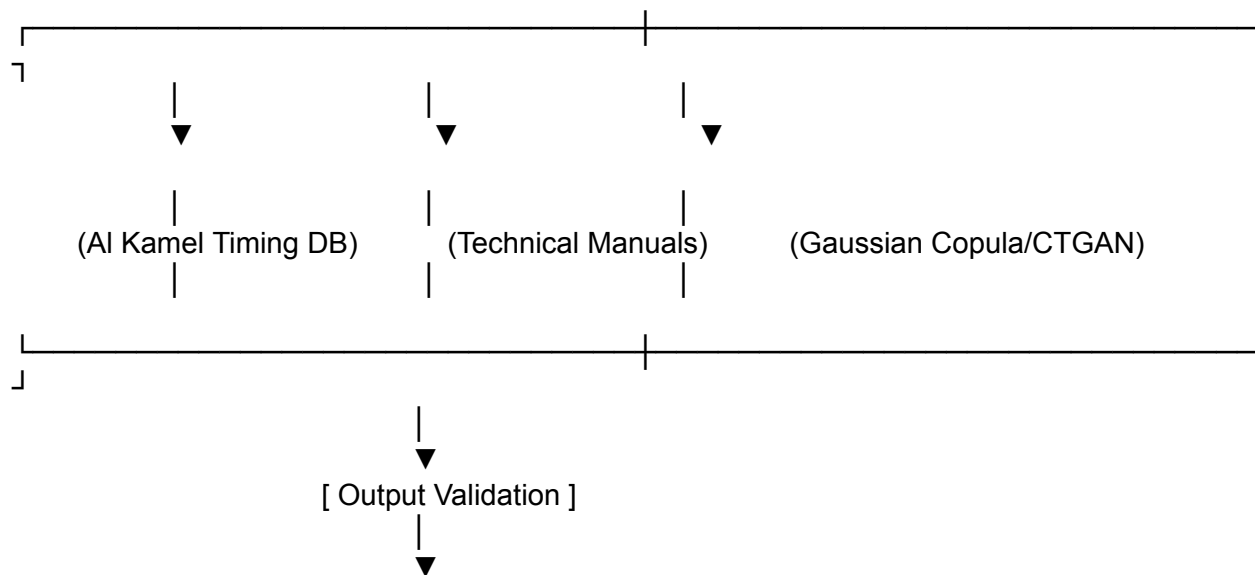


[Input Guardrail Node]



[Orchestrator Agent Node]





The state machine employs a multi-agent graph where individual nodes are assigned specialized strategic and diagnostic sub-tasks. This graph-based structure allows the system to run recursive loops, enabling the local assistant to validate database queries or cross-reference diagnostic suggestions against technical schemas before delivering recommendations to the engineers.

Data Ingestion and Database Schema Design

The strategy assistant's predictive models rely on historical and live-stream timing data parsed from AI Kamel Systems, the primary timing and data provider for WEC and IMSA. Scraping and processing this data requires strict compliance with copyright and ownership terms. AI Kamel Systems retains explicit ownership of all timing materials, meaning automated scraping and storage must remain strictly confined to the team's internal strategic operations.

Historical timing logs are stored as highly detailed CSV analysis sheets, scraped from the AI Kamel cloud publisher servers. This historical data is programmatically parsed and mapped into a local SQLite timing database. During live sessions, the ingestion engine switches from static file parsing to a low-latency TCP/IP socket listening to the Enhanced RMon Protocol broadcasted by the trackside timing servers.

AI Kamel Data Channel	Server Location / Endpoint	Protocol Format	Operational Utility
WEC Cloud Publisher	fiawec.datapublisher.alkamelcloud.com	Structured CSV Files	Programmatic scraping of historical lap, sector, and weather metrics.
WEC On-Track Server	timing3.aks	TCP/IP Enhanced RMon Stream	Real-time crossing packet decoding for live race strategy updates.
IMSA Cloud Publisher	imsa.datapublisher.alkamelcloud.com	Structured CSV Files	Programmatic scraping of historical practice, qualifying, and race

AI Kamel Data Channel	Server Location / Endpoint	Protocol Format	Operational Utility
			sessions.
IMSA On-Track Server	timing-data.aks	TCP/IP Enhanced RMon Stream	Real-time crossing packet decoding for live race strategy updates.

The live stream transmits timing packets in a standardized structure: where `\angle\text{SOR}\rangle` represents the Start of Record character (\$), `\angle\text{Sep}\rangle` is the comma separator (0x2C), and `\angle\text{EOR}\rangle` is the carriage return and line feed sequence (0x0D, 0x0A). The timing parser decodes these packets on the fly to monitor track conditions and competitor pacing:

- **Competitor Crossing (Command A):** Triggers instantaneous sector calculation and theoretical best update.
- **Track Status (Command B):** Signals safety car, full-course yellow (FCY), or red flag state transitions.
- **Pit Lane Status (Command L):** Tracks when a car enters or exits the pitlane to log stationary tyre/fuel service time.
- **Competitor Entry (Command `[span_59](start_span)[span_59](end_span)[span_64](start_span)[span_64](end_span)COMP`):** Updates registration, driver swaps, and class assignments on the fly.

Raw Socket Packet -> Regex Parser -> Token Validator -> DB Cache -> Strategy Engine

The decoded data is structured into a local SQLite database containing two main tables, laps and driver_elo, enabling rapid querying and strategic planning.

SQLite Table	Field Name	Data Type	Engineering Application	Source
laps	series_code	VARCHAR	Identifies championship parameters (wec, imsa) to apply correct sporting regulations.	
	class	VARCHAR	Identifies specific competitor vehicle and class (GTP, LMH, LMP2, GTD).	
	driver_name	VARCHAR	Driver identification to map degradation profiles and ELO ratings.	
	lap_time	FLOAT	Raw duration of the lap, converted to seconds for mathematical optimization.	

SQLite Table	Field Name	Data Type	Engineering Application	Source
	S1 to S9	FLOAT	Sector and microsector times used to isolate aerodynamic and mechanical grip profiles.	
	PIT_TIME	FLOAT	Stationary and pitlane residency time used to calculate pitstop delta penalties.	
	track_temp_f	FLOAT	Environmental tracking for thermal tyre wear regressions.	
	raining	BOOLEAN	Status flag to trigger alternate dynamic rain strategies and compound thresholds.	
driver_elo	driv [span_79](start _span)[span_79](e nd_span)er_id	VARCHAR	Normalized unique identifier stable across name variants.	
	elo_before	INTEG [span_80](s tart_span)[span_8 0](end_span)ER	Rating before event to assess baseline driver pace and consistency.	
	cumulative_laps	l [span_81](start_s pan)[span_81](end _span)NTEGER	Career laps driven in the specific class to track driver experience.	

Vehicle Dynamics and Powertrain Formulations

To deliver accurate strategic recommendations, the chatbot's inference layers rely on mathematical models of vehicle dynamics, fuel burn-off, and tyre degradation.

Fuel Burn-off and Mass Correction

The instantaneous mass of the vehicle decreases linearly with each lap as the internal combustion engine burns fuel :

where $M_{\text{vehicle}}(t)$ is the current vehicle mass, M_{initial} is the starting mass, and $\dot{m}_{\text{fuel}}(\tau)$ is the fuel consumption rate per lap (e.g., 1.84 kg/lap in typical

high-downforce trims). Because vehicle weight directly dictates lateral and longitudinal grip, a lighter vehicle inherently achieves faster lap times. The fuel weight correction is modeled linearly:

The standard fuel weight penalty coefficient α_{fuel} is approximated at $0.03 \text{ seconds per kilogram}$ of fuel burned (or 0.3 s per 10 kg), representing a significant performance gain over a long endurance stint.

Non-Linear Tyre Degradation

Tyre performance decreases with cumulative thermal and mechanical stress. Correct modeling requires isolating tyre wear from the fuel burn-off effect. The degradation curve exhibits a multi-phase characteristic:

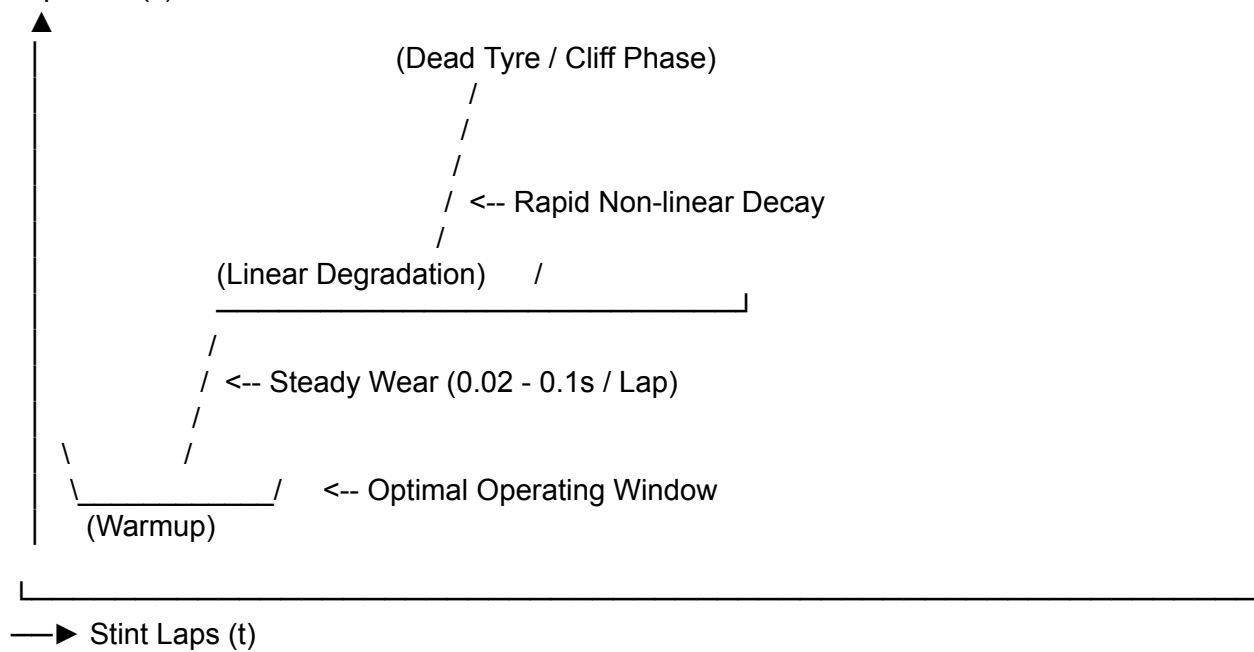
The baseline tyre wear $f(\text{compound}, t)$ is modeled as a polynomial or state-space regression:

where t represents the cumulative laps on the tyre set. In professional endurance settings, the degradation is segmented into clear operating windows:

1. **Warmup Phase:** The tyre compound is brought up to core pressure and temperature, where $\beta_{\text{tyre}}(t) \leq 0$, resulting in a brief increase in mechanical grip.
2. **Steady-State Phase:** Linear wear occurs, leading to a predictable degradation rate of approximately $0.02 \text{ to } 0.1 \text{ s/lap}$ depending on the selected compound (Soft, Medium, Hard).
3. **Thermal Drop-Off Phase:** Once the tread thickness drops below a critical threshold, the heat rejection capacity of the tyre is compromised, leading to core overheating. This manifests as a non-linear performance drop-off:

Under severe thermal drop-off, the car can lose up to $3 \text{ to } 4 \text{ seconds per lap}$ within the span of three laps, even when visual indicators show substantial rubber remaining, highlighting a significant physical-to-telemetry mismatch.

Lap Time (s)



The fuel burn-off effect can easily mask tyre degradation in raw timing sheets. This masking

effect introduces a major strategic risk: a car might maintain consistent lap times, giving the impression that its tyres are healthy, while actually approaching the critical thermal drop-off. If the strategist delays the pit stop to attempt an overcut, the driver can suddenly drop off the performance "cliff," losing several seconds per lap and neutralizing any strategic advantage.

Hypercar and Grand Touring Prototype Hybrid System Specifications

The strategy assistant must handle the architectural differences between LMH and LMDh classes under strict Balance of Performance (BoP) rules. While LMH allows bespoke manufacturer hybrid systems, LMDh mandates a standardized rear-wheel hybrid spine.

Vehicle Model	Category	Engine Architecture	Hybrid Axle	MGU Peak Output / Regen	Unique System Characteristics
Toyota GR010	LMH	3.5\text{L} Twin-Turbo V6	Front Axle	200\text{ kW} Deployment / 200\text{ kW} Regen	Front-axle recovery; deployment restricted by speed threshold (e.g., 190\text{ kph}).
Ferrari 499P	LMH	3.0\text{L} Twin-Turbo V6	Front Axle	200\text{ kW} Deployment / 200\text{ kW} Regen	Front MGU driven by a bespoke 6-phase switching inverter integrated into the battery pack.
Aston Martin Valkyrie	LMH	6.5\text{L} Naturally Aspirated V12	None	None	Pure mechanical drive with zero hybrid energy recuperation or deployment.
Porsche 963	LMDh	4.6\text{L} Twin-Turbo V8	Rear Axle	50\text{ kW} Deployment / 180\text{ kW} Regen	Standardized spine; engine based on 918 Spyder with twin Van der Lee turbochargers in a hot vee.
BMW M Hybrid V8	LMDh	4.0\text{L} Twin-Turbo V8	Rear Axle	50\text{ kW} Deployment / 180\text{ kW}	Standardized spine; utilizes a modified

Vehicle Model	Category	Engine Architecture	Hybrid Axle	MGU Peak Output / Regen	Unique System Characteristics
				Regen	DTM-derived V8 engine.
Cadillac V-Series.R	LMDh	5.5\text{L} Naturally Aspirated V8	Rear Axle	50\text{ kW} Deployment / 180\text{ kW} Regen	Standardized spine; uses high-displacement naturally aspirated power unit with immediate throttle response.
Genesis LMDh	LMDh	Twin-Turbo V6 / V8	Rear Axle	50\text{ kW} Deployment / 180\text{ kW} Regen	Standardized spine; upcoming premium performance program using the unified Bosch/WAE/Xtrac hybrid package.

These architectural differences directly impact tyre management. Because LMH platforms are restricted to deploying hybrid power on the front axle only above a speed threshold (typically 190\text{ kph}), they operate as purely rear-wheel-drive vehicles out of low-speed corners. Consequently, the rear tyres must handle the entire torque output of the internal combustion engine during low-speed acceleration, accelerating rear tyre wear.

In contrast, LMDh platforms can deploy their 50\text{ kW} hybrid power on the rear axle from 0\text{ kph}, allowing the motor generator unit to assist traction out of slow corners, which helps mitigate wheelspin and stabilize rear tyre temperatures.

Low-Speed Acceleration (< 190 kph):

LMH: ---> (Accelerates rear tyre wear)

LMDh: ---> (Mitigates wheelspin & stabilizes temps)

The standardized LMDh hybrid spine is built around tightly integrated components developed by Bosch, Williams Advanced Engineering (WAE), and Xtrac :

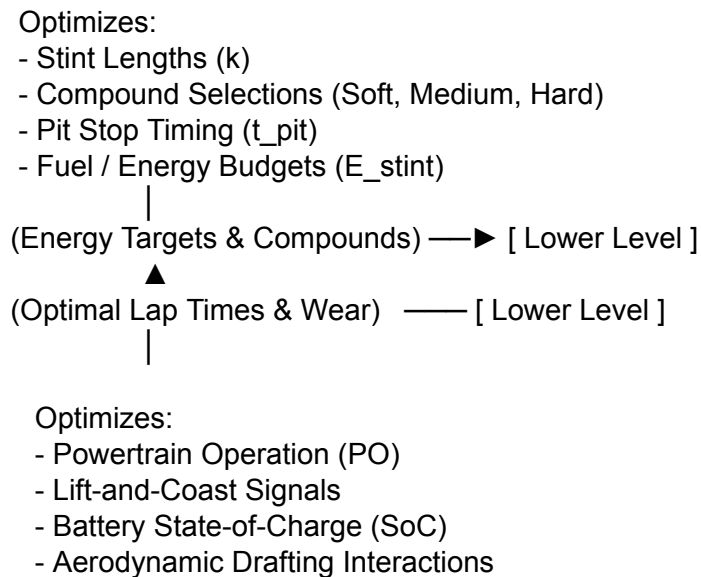
- **Bosch Motor Generator Unit (MGU):** A high-voltage, liquid-cooled electric machine mounted inside the Xtrac gearbox bell housing. It rotates up to 20,000\text{ rpm}, delivering 50\text{ kW} (67\text{ hp}) of continuous propulsion and recovering up to 180\text{ kW} (241\text{ hp}) under braking.
- **Bosch Motor Control Unit (MCU):** An advanced inverter based on Generation 4 silicon carbide (SiC) semiconductor technology. It regulates high-voltage AC-DC conversion, manages core thermal limits, and coordinates regenerative torque split.
- **Williams Advanced Engineering Battery:** An 800\text{ V} lithium-ion energy storage system with an energy capacity of 1.35\text{ kWh}. It is designed to handle very high charge and discharge rates, allowing rapid power transfer within a compact, lightweight

packaging envelope.

- **Bosch Electric Brake System (EBS):** A brake-by-wire system that automatically balances mechanical hydraulic braking pressure with electromagnetic MGU deceleration on the rear axle.

Stochastic Pit Stop Strategy and Stint Planning Engine

The strategic core of the assistant utilizes a bi-level optimization framework to compute the race strategy that minimizes total race duration.



Lower-Level Dynamic Control

The lower level solves a multi-agent, game-theoretic optimal control problem to calculate minimum-lap-time trajectories while accounting for aerodynamic interactions and powertrain limits :

subject to the physical limit constraints of the vehicle dynamics, motor torque curves, battery thermal limits, and aerodynamic interactions (such as slipstreaming and dirty air).

Following in a competitor's wake significantly reduces aerodynamic drag, which lowers fuel and electrical energy consumption, extending the stint length and saving critical pit stop time.

However, this "dirty air" also reduces front-axle downforce, causing the tyres to slide and wear more rapidly. The lower-level controller balances these factors to determine optimal lift-and-coast profiles and battery state-of-charge targets.

Upper-Level Stint Allocation

The upper-level scheduler uses the optimized lap-time maps to compute the global stint progression. The problem is formulated as a Mixed-Integer Linear Programming (MILP) model to determine stint allocations, compound choices, and pit-stop timing :

subject to:

where $|L_k|$ represents the length of stint k , T_{lap} is the fuel-and-wear-corrected lap time, and $T_{\text{pit}}(k)$ is the pit lane time penalty, which includes driver swaps, tyre changes, and refueling durations.

To handle real-time uncertainty, the scheduler implements a Stochastic Dynamic Programming (SDP) formulation. This formulation evaluates the transition probabilities of track incidents (such as full-course yellows or safety cars) to dynamically optimize the pit strategy :

where s_t represents the current state (lap, tyre wear, fuel level, weather, track status), a_t is the strategic action (stay out or pit for a specific compound), and $P(s_{t+1} \mid s_t, a_t)$ is the transition probability matrix.

This system also factors in multiclass traffic management. For example, when an LMP2 or GTP car catches the back of the slower GT3 pack, its lap times drop due to traffic. Pitting three to four laps after encountering this traffic drops the car into clear air, allowing the driver to perform an overcut and leapfrog competitors delayed by the traffic.

Overcoming Data Sharing Restrictions via Synthetic Telemetry

Because manufacturers restrict proprietary telemetry mappings, the strategy assistant must run on synthetic telemetry that mimics the statistical relationships of actual race data without exposing sensitive intellectual property. To generate these realistic datasets, the assistant integrates the Synthetic Data Vault (SDV) library alongside schema-driven generation engines. The framework uses a GaussianCopulaSynthesizer or a deep learning-based Conditional Tabular GAN (CTGAN) to learn the statistical structure of a non-sensitive data sample and generate highly realistic synthetic copies.

```
import pandas as pd
from sdv.metadata import SingleTableMetadata
from sdv.single_table import GaussianCopulaSynthesizer
```

```
def generate_synthetic_telemetry(raw_sample_df: pd.DataFrame, num_rows: int = 5000) ->
pd.DataFrame:
```

```
    """
    Learns statistical correlations from a non-sensitive telemetry sample
    and generates representative synthetic data.
    """
```

```
    # Define metadata schema for the target parameters
    metadata = SingleTableMetadata()
    metadata.detect_from_dataframe(data=raw_sample_df)
```

```
    # Train the synthesizer on the sample data
    synthesizer = GaussianCopulaSynthesizer(metadata)
    synthesizer.fit(data=raw_sample_df)
```

```
    # Generate synthetic telemetry preserving correlations (e.g., Throttle vs. MGU Output)
    synthetic_data = synthesizer.sample(num_rows=num_rows)
    return synthetic_data
```

To maintain database relationships, tables are generated in topological dependency order. Parent tables (such as session and driver metadata) are constructed first, and child tables (such as individual lap telemetry records) sample from the parent pool, preventing referential integrity errors.

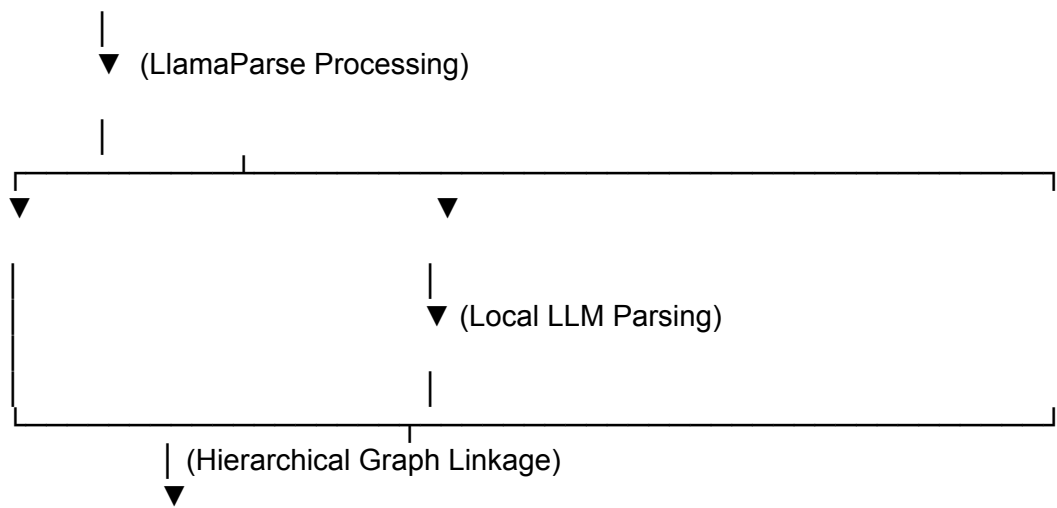
This method ensures that sensitive manufacturer details—such as the relationship between throttle percentage, electric brake regeneration torque, and inverter cooling temperatures—are preserved statistically for model training without exposing the actual proprietary maps.

Local Advanced RAG and Agentic SQL Execution

The chatbot interface converts natural language queries from the pit wall into structured database queries and manual extractions.

Advanced Diagnostic RAG with LlamaParse

To support technical troubleshooting (such as resolving sensor issues or hybrid system fault codes), the chatbot integrates an Advanced Retrieval-Augmented Generation (RAG) pipeline. Because manufacturer manuals are dense with nested tables, wiring schematics, and sensor code ranges, standard OCR-based parsing is inadequate. The ingestion pipeline utilizes LlamaParse to convert technical documents into semi-structured Markdown. It then extracts tabular arrays, generates summaries for each table using a local model, and builds a hierarchical index where summary nodes link directly to the source technical chunks.



When an engineer inputs a diagnostic query (e.g., *"How do we override MGU thermal fault code F33 on the front axle?"*), the recursive retriever first matches the table summaries and then pulls the exact step-by-step schematics and override sequences.

Stateful LangGraph Text-to-SQL Workflow

For structured timing analysis, the orchestrator routes natural language questions through a stateful LangGraph agentic pipeline. This workflow converts the engineer's query into a precise SQL statement, executes it against the local database, and returns the formatted results.

```
import sqlite3
from typing import
```

```

Li[span_232](start_span)[span_232](end_span)[span_236](start_span)[span_236](end_span)ter
al, Dict, Any
from langgraph.graph import StateGraph, START, END
from langchain_core.tools import tool

```

```

# Define state parameters for orchestration

```

```

class TeamState(dict):
    messages: list
    sql_query: str
    query_result: str
    validated: bool

```

```

@tool

```

```

def validate_and_execute_query(sql_query: str) -> str:
    """

```

```

    Validates and executes the generated SQL query against the SQLite database.
    Includes guardrails to block destructive commands (e.g., DROP, DELETE).
    """

```

```

    # Strict security check to block DML injection commands
    forbidden_keywords = ["drop", "delete", "insert", "update", "alter", "truncate"]
    if any(keyword in sql_query.lower() for keyword in forbidden_keywords):
        return "Security Violation: Unauthorized database modification command blocked."

```

```

    connection = sqlite3.connect("trackside_timing.db")
    cursor = connection.cursor()
    try:
        cursor.execute(sql_query)
        results = cursor.fetchall()
        return str(results)
    except sqlite3.Error as e:
        return f"Database Error: {str(e)}"
    finally:
        connection.close()

```

This workflow implements input and output guardrails. Input guardrails detect potential prompt injections and strip malicious commands. Output guardrails intercept the generated SQL, blocking destructive data manipulation language statements (such as DROP, DELETE, or INSERT) to protect the database during live operations.

Technical Implementation Blueprint inside the Anaconda and VS Code Workspace

The cognitive strategy and diagnostics assistant is designed to be deployed and executed locally on trackside engineering hardware running VS Code or Anaconda Navigator. This setup ensures that the entire system runs offline, protecting proprietary manufacturer data. Trackside Hardware (Conda Env) -> local Ollama API (11434) -> Qwen/Mistral -> Streamlit App

Step 1: Conda Environment Initialization

To isolate dependencies and avoid conflicts with other software on the device, the engineering team must initialize a virtual environment running Python 3.10. Run the following commands in the terminal or Anaconda Prompt:

```
# Create and activate the conda environment
conda create -n strategy_assistant python=3.10 -y
conda activate strategy_assistant
```

Step 2: Dependency Installation

Install the required machine learning, database, and front-end libraries :

```
# Install core database, ML, and agentic libraries
pip install sdv pandas numpy matplotlib
pip install
langchain_community langchain-ollama langgraph
pip install llama-index llama-parse chromadb
pip install streamlit
uvicorn
```

Step 3: Local LLM Configuration

Download and start the Ollama service to manage the quantized local models :

```
# Pull the required models locally
ollama pull qwen2.5-coder:7b
ollama pull mistral:7b
```

These models run locally via Port 11434, providing secure, low-latency inference without sending any data over the internet.

Step 4: Streamlit Front-End Orchestration

Create a local Python file named app.py in your VS Code workspace to serve as the team's interactive portal. This script builds the front-end interface, connects to the local database, and queries the local LLM :

```
import streamlit as str_ui
import pandas as pd
import sqlite3
from langchain_community.llms import Ollama

# Page setup
str_ui.set_page_config(page_title="Cognitive Strategy Assistant", layout="wide")
```

```

str_ui.title("Cognitive Strategy & Diagnostics Assistant")

# Initialize the local LLM connection
local_llm = Ollama(model="qwen2.5-coder:7b", base_url="http://localhost:11434")

# Sidebar configurations
str_ui[span_10](start_span)[span_10](end_span)[span_15](start_span)[span_15](end_span).side
bar.header("Trackside System Status")
stint_lap = str_ui.sidebar.number_input("Current Stint Lap", min_value=1, max_value=45,
value=12)
track_temp = str_ui.sidebar.slider("Track Temperature (°F)", 60, 140, 95)

# Primary input area
user_query = str_ui.text_input("Consult the strategy and diagnostics assistant:")

if user_query:
    with
str_ui.spinner("An[span_30](start_span)[span_30](end_span)[span_32](start_span)[span_32](e
nd_span)alyzing telemetry and manual databases..."):
    # Process the query using the local LLM
    response = local_llm.invoke(user_query)
    str_ui.write("### Assistant Response")
    str_ui.info(response)

```

To launch the interface, run the following command in the active conda terminal:

```
streamlit run app.py
```

The local interface will open automatically in your browser at <http://localhost:8501>, providing a secure, offline environment for trackside strategy and diagnostics.

Strategic and Diagnostic Recommendations

To maximize the performance of WEC and IMSA campaigns, the engineering team should implement the following procedures:

1. **Local Edge Deployment:** Run all database and inference operations on a local UNIX server on the engineering gantry. Configure a local network endpoint (<http://localhost:11434>) to execute local SLMs (such as Qwen 2.5 Coder), ensuring complete protection of manufacturer IP.
2. **Automate Ingestion:** Build a robust Python scraper to fetch historical timing sheets from Al Kamel Systems, storing them in a local SQLite database. For live sessions, stream timings directly from the trackside Enhanced RMon socket to enable real-time tactical adjustments.
3. **Isolate Tyre Performance:** Apply the linear fuel mass correction formula ($\approx 0.03 \text{ s/kg}$) to the raw live lap times. This filters out the fuel burn-off effect, allowing the model to isolate true tyre degradation and detect when the tires are approaching a critical thermal drop-off.
4. **Optimize Pit Windows:** Configure the upper-level strategic planner to run real-time

simulations during on-track disruptions (such as full-course yellows or safety cars). This allows the team to evaluate the trade-offs of pitting early for fresh compounds versus staying out to maintain track position.

5. **Synthesize Missing Data:** Address manufacturer data sharing restrictions by generating synthetic telemetry via SDV. This creates highly realistic datasets that preserve critical statistical correlations for model training without exposing proprietary vehicle configurations.
6. **Pre-Index Manuals:** Run all technical specifications, sporting regulations, and hybrid troubleshooting guidelines through the LlamaParse engine. Index the resulting Markdown chunks into a local vector store to enable fast, offline diagnostic lookup during critical sessions.

By establishing this local, mathematically grounded framework, the team can leverage high-resolution timing data and cognitive diagnostics. This provides a distinct strategic advantage, enabling faster, more accurate decisions while keeping proprietary manufacturer configurations completely secure.

Works cited

1. 2026 IMSA SPORTING REGULATIONS and SSR IWSC REDLINE PUBLICATION 111125 V1, <https://www.imsa.com/wp-content/uploads/sites/32/2025/11/11/2026-IMSA-SPORTING-REGULATIONS-and-SSR-IWSC-Redline-111125.pdf>
2. Le Mans Daytona hybrid system | Bosch Motorsport, <https://www.bosch-motorsport.com/technology-and-innovation/product-highlights/hybrid-system-lmdh/>
3. How to Get the Most Out of a Hypercar Hybrid System - Racecar Engineering, <https://www.racecar-engineering.com/articles/sportscar/how-to-get-the-most-out-of-a-hypercar-hybrid-system/>
4. Hybrid 101: Learn More about How LMDh Hybrid Power Works | IMSA, <https://www.imsa.com/news/2022/04/06/hybrid-101-learn-more-about-how-lmdh-hybrid-power-works/>
5. Building a Local AI Agent with Ollama and LangChain: A Practical Guide - DEV Community, <https://dev.to/midas126/building-a-local-ai-agent-with-ollama-and-langchain-a-practical-guide-563e>
6. Building AI Agents with Local Small Language Models - MachineLearningMastery.com, <https://machinelearningmastery.com/building-ai-agents-with-local-small-language-models/>
7. IMSA - Timing Results - Al Kamel Systems, <http://imsa.alkamelsystems.com/>
8. LangChain Project 1: Build a 100% Local AI Dev Helper | Llama 3 + Ollama, <https://www.youtube.com/watch?v=4glfTerh3vs>
9. Yelp Restaurant data scraping using python, scrapy spider - GitHub, <https://github.com/farukalamai/yelp-scraper-scrapy-python>
10. rembertdesigns/pit-stop-simulator: An interactive F1 race simulator using Reinforcement Learning (PPO, Q-learning) and Streamlit to optimize pit stop strategies based on dynamic conditions. - GitHub, <https://github.com/rembertdesigns/pit-stop-simulator>
11. Building a Text-to-SQL Assistant with LangGraph: From Prompt to Production | by Akhil Golla, <https://medium.com/@akhildevang/building-a-text-to-sql-assistant-with-langgraph-from-prompt-to-production-2dd2ec0ac37f>
12. Build a LangGraph SQL Agent with Mistral Medium 3 | IBM, <https://www.ibm.com/think/tutorials/build-sql-agent-langgraph-mistral-medium-3-watsonx-ai>
13. I Built a Local AI Agent with Gemma 4 — Runs Fully Offline, <https://www.youtube.com/watch?v=VhjsHBCqwrQ>
14. How to Run a Local AI Agent with Ollama, Qwen 3, and LangGraph, <https://www.youtube.com/watch?v=MAZUI4jGIU8>
15. Build a custom SQL agent - Docs by LangChain, <https://docs.langchain.com/oss/python/langgraph/sql-agent>

16. Welcome to LlamaIndex ! | Developer Documentation - LlamaParse, <https://developers.llamaindex.ai/python/framework/> 17. LangGraph: SQL Query Generation - Kaggle, <https://www.kaggle.com/code/ksmooi/langgraph-sql-query-generation> 18. Al Kamel Systems: Home, <https://alkamelsystems.com/> 19. WEC/notebooks/Simple demo.ipynb at master · f1datajunkie/WEC - GitHub, <https://github.com/f1datajunkie/WEC/blob/master/notebooks/Simple%20demo.ipynb> 20. Where to get all laptime data from an IMSA race? : r/wec - Reddit, https://www.reddit.com/r/wec/comments/18l7cvz/where_to_get_all_laptime_data_from_an_imsa_race/ 21. Alkamel V2 - HH Timing Help, <https://help.hhtiming.com/timekeeper-specific-info/alkamel-v2/> 22. tobi/imsa_data - GitHub, https://github.com/tobi/imsa_data 23. Formats description/documentation for Al Kamel Systems CSV and Replays files? - Reddit, https://www.reddit.com/r/IMSARacing/comments/10wdex5/formats_descriptiondocumentation_for_al_kamel/ 24. Porsche 963 LMDh/GTP Hybrid Prototype Technical Insight - Racecar Engineering, <https://www.racecar-engineering.com/articles/porsche-963-lmdh-insight/> 25. TracingInsights-Archive/2025: 2025 Public F1 telemetry Data - GitHub, <https://github.com/TracingInsights-Archive/2025> 26. [Data Analysis] Building an ML Model for Race Pace: Is Fuel Burn-off masking Tyre Degradation in my correlation matrix? (+ Anomaly in Rain Data) : r/F1Technical - Reddit, https://www.reddit.com/r/F1Technical/comments/1p2v25h/data_analysis_building_an_ml_model_for_race_pace/ 27. Formula 1 | Bahrain Grand Prix Preview - Racecar Engineering, <https://www.racecar-engineering.com/news/formula-1bahrain-grand-prix-preview/> 28. Building a race simulator | f1metrics - WordPress.com, <https://f1metrics.wordpress.com/2014/10/03/building-a-race-simulator/> 29. How Tyre Degradation Impacts Race Strategy: Key Factors - Catapult Sports, <https://www.catapult.com/blog/how-tyre-degradation-affects-race-strategy> 30. Tire Degradation in F1 Using Real Race Data – Flat Trends, No Clear Curve - Reddit, https://www.reddit.com/r/F1Technical/comments/1k6nzct/tire_degradation_in_f1_using_real_race_data_flat/ 31. Data-driven pit stop decision support for Formula 1 using deep learning models - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12626961/> 32. Is there a mathematical way to calculate tyre degradation? : r/F1Technical - Reddit, https://www.reddit.com/r/F1Technical/comments/17s8kbs/is_there_a_mathematical_way_to_calculate_tyre/ 33. (PDF) A state-space approach to modeling tire degradation in formula 1 racing, https://www.researchgate.net/publication/404228712_A_state-space_approach_to_modeling_tire_degradation_in_formula_1_racing 34. Is tire wear linear over time? : r/F1Technical - Reddit, https://www.reddit.com/r/F1Technical/comments/rawzw6/is_tire_wear_linear_over_time/ 35. Endurance Hall LMDh- Tyre Degradation Mismatch and AI Behavior | Straight4 Studios Community Hub, <https://hub.straight4.com/index.php?threads/endurance-hall-lmdh-tyre-degradation-mismatch-and-ai-behavior.1323/> 36. Pit strategy for 1 stop endurance race with good pace : r/iRacing - Reddit, https://www.reddit.com/r/iRacing/comments/1nz8o6u/pit_strategy_for_1_stop_endurance_race_with_good/ 37. How to Make the Most of Le Mans Ultimate's Hybrid Systems - Trophi.ai, <https://www.trophi.ai/post/how-to-make-the-most-of-le-mans-ultimates-hybrid-systems> 38. The Porsche LMDh prototype enters active test phase, <https://newsroom.porsche.com/en/2022/motorsports/porsche-lmdh-prototype-test-phase-hybrid-drive-27200.html> 39. The electric motor (MGU) by Bosch for the LMDh prototypes provides up to 200 kW of power for high performance and reliability.,

<https://www.bosch-presse.de/pressportal/us/en/image-19971.html> 40. Inside the Porsche 963: Technical Details Revealed - Sportscar365, <https://sportscar365.com/lemans/wec/inside-the-porsche-963-technical-details-revealed/> 41. The technology of the new Porsche 963 hypercar/GTP prototype in detail, <https://newsroom.porsche.com/en/2023/motorsports/porsche-hypercar-gtp-prototype-963-technology-31043.html> 42. Bosch hybrid system for motorsports debuts at Rolex 24 At Daytona, <https://us.bosch-press.com/pressportal/us/en/press-release-19968.html> 43. Competitor-aware Race Management for Electric Endurance Racing - arXiv, <https://arxiv.org/html/2603.28286v1> 44. [2603.28286] Competitor-aware Race Management for Electric Endurance Racing - arXiv, <https://arxiv.org/abs/2603.28286> 45. A simulation framework for Formula 1 race strategy based on pit-stop optimization, https://optimization-online.org/wp-content/uploads/2026/02/Pit_Stop_Strategy_Optimization_Model.pdf 46. Maximum-distance Race Strategies for a Fully Electric Endurance Race Car | Request PDF - ResearchGate, https://www.researchgate.net/publication/361394379_Maximum-distance_Race_Strategies_for_a_Fully_Electric_Endurance_Race_Car 47. Model Predictive Control Strategies for Electric Endurance Race Cars Accounting for Competitors' Interactions | Request PDF - ResearchGate, https://www.researchgate.net/publication/381568118_Model_Predictive_Control_Strategies_for_Electric_Endurance_Race_Cars_Accounting_for_Competitors'_Interactions 48. Formula-E Multi-Car Race Strategy Development—A Novel Approach Using Reinforcement Learning | Request PDF - ResearchGate, https://www.researchgate.net/publication/380411914_Formula-E_Multi-Car_Race_Strategy_Development-A_Novel_Approach_Using_Reinforcement_Learning 49. Towards Learning-Based Formula 1 Race Strategies - arXiv, <https://arxiv.org/html/2512.21570v1> 50. Pit Stop Strategy in Endurance Sim Racing - Fanatec, <https://www.fanatec.com/us/en/explorer/games/gaming-tips/pit-stop-strategy-in-endurance-sim-racing/> 51. Optimizing pit stop strategies in Formula 1 with dynamic programming and game theory - IDEAS/RePEc, <https://ideas.repec.org/a/eee/ejores/v319y2024i3p908-919.html> 52. GitHub - sdv-dev/SDV: Synthetic data generation for tabular data, <https://github.com/sdv-dev/sdv> 53. Building the Best Synthetic Data Generator in Python for 2026: Why I Am Building Misata and How to Use It | by Muhammed Rasin - Towards AI, <https://pub.towardsai.net/building-the-best-synthetic-data-generator-in-python-for-2026-why-i-am-building-misata-and-how-to-3cedd65242ed> 54. machine learning - best way to create Synthetic data generation, <https://datascience.stackexchange.com/questions/134428/best-way-to-create-synthetic-data-generation> 55. Advanced RAG for Complex Documents | PDF | Parsing | Computing - Scribd, <https://www.scribd.com/presentation/825852585/LlamaIndex-Talk-Data-AI-Summit-2024> 56. Free Advanced RAG Certification course with Actueloop and LlamaIndex, <https://www.llamaindex.ai/blog/join-thousands-in-our-free-advanced-rag-certification-created-with-actueloop-ad63f24f27bb> 57. Advanced RAG Cheat Sheet And Recipes Guide | LlamaIndex, <https://www.llamaindex.ai/blog/a-cheat-sheet-and-some-recipes-for-building-advanced-rag-803a9d94c41b>